

13 — Installer le projet avec Docker sur un autre PC (Windows)

But : que le projet tourne à **l'identique** chez un camarade, sans WAMP, sans Go, sans MySQL installé. **Docker fait tout.**
Résultat attendu : site accessible sur `http://localhost:8088` .

Étape 0 — Prérequis à installer (une fois)

1. **Docker Desktop** (Windows) : <https://www.docker.com/products/docker-desktop/>
2. À l'installation, laisser coché **WSL 2** (recommandé).
3. Redémarrer le PC si demandé.
4. Lancer Docker Desktop et **attendre l'icône verte** (« Engine running »).
5. **Git** (pour récupérer le projet) : <https://git-scm.com/download/win>
6. (Alternative sans Git : copier tout le dossier du projet sur une clé USB.)

Vérifier que Docker fonctionne (dans un terminal PowerShell ou Git Bash) :

```
docker --version
docker compose version
```

Étape 1 — Récupérer le projet

Option A — avec Git :

```
cd C:/Users/NOM/Desktop
git clone <URL_DU_DEPOT> upcycleconnect
cd upcycleconnect
```

Option B — sans Git : copier le dossier complet du projet (celui qui contient `docker-compose.yml` , `API/` , `Frontend/` , `db/`) sur le PC, puis ouvrir un terminal **dans ce dossier**.

⚠ Il faut que le dossier contienne bien : `docker-compose.yml` , `API/Dockerfile` , `Frontend/Dockerfile` , `db/init.sql` .

Étape 2 — Lancer le projet

Dans le terminal, à **la racine du projet** :

```
WEB_PORT=8088 docker compose up -d --build
```

- La **première fois** : le build prend **2 à 5 minutes** (téléchargement des images + compilation Go). C'est normal.
- `WEB_PORT=8088` évite le conflit avec le port 80.

Sur **PowerShell** (si `WEB_PORT=8088 ...` ne marche pas), faire : `powershell $env:WEB_PORT=8088 ; docker compose up -d --build`

Étape 3 — Vérifier que tout tourne

```
docker ps
```

On doit voir **3 conteneurs Up** : `uc_mysql` (healthy), `uc_backend` , `uc_frontend` .

Puis ouvrir dans le navigateur : **http://localhost:8088** La page d'accueil doit s'afficher. Se connecter avec un compte de test (voir `09_COMPTES_TEST.md`).

La base de données se remplit **automatiquement** au 1er démarrage via `db/init.sql` — rien à importer à la main.

Étape 4 — Arrêter / relancer

```
docker compose stop      # arrêter (garde les données)
docker compose start      # relancer
docker compose down       # arrêter et supprimer les conteneurs (garde les volumes/données)
```

Dépannage (les 5 problèmes classiques)

Problème	Cause	Solution
« port is already allocated » / 8088 ou 8081 occupé	Un autre programme utilise le port	Changer le port : <code>WEB_PORT=8090 docker compose up -d</code> . Pour le 8081, fermer WAMP/un <code>go run</code> qui tourne.
Docker Desktop pas démarré (error during connect ... pipe)	L'Engine n'est pas lancé	Ouvrir Docker Desktop , attendre l'icône verte, réessayer.
<code>uc_mysql</code> reste « unhealthy »	Ancien volume MySQL incompatible	<code>docker compose down -v</code> puis <code>docker compose up -d</code> (réinitialise la base).
Le site ne charge pas / erreurs API	Front lancé seul sans backend	Toujours lancer la stack complète : <code>docker compose up -d</code> .
Modif invisible dans le navigateur	Cache	Ctrl+Shift+R .

Reset complet (repartir de zéro proprement)

```
docker compose down -v      # supprime conteneurs + volumes (efface la base)
WEB_PORT=8088 docker compose up -d --build
```

Accéder à la base avec phpMyAdmin (interface web)

Un service **phpMyAdmin** est inclus dans `docker-compose.yml` . Après `docker compose up -d` :

- Ouvrir : **http://localhost:8082**
- Utilisateur : `upcycle` / Mot de passe : `upcyclePass123` (base `pa2026`)
- ou `root` / `rootSecret123` (accès total)

Il se connecte à MySQL en interne (`PMA_HOST=mysql`) → aucun conflit avec le 3306 de WAMP. Changer le port si besoin :
`PMA_PORT=8090 docker compose up -d` .

Alternative sans phpMyAdmin (SQL direct) :

```
docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026
```

⚠ Les modifs de base vivent dans le volume `mysql_data` : conservées après `stop/start/down` , mais **effacées par** `docker compose down -v` (réimport de `db/init.sql`). Pour une modif permanente pour tout le monde → la reporter dans `db/init.sql` .

Sur le serveur de prod (docker-compose-prod.yml)

- phpMyAdmin utilise l'image publique `phpmyadmin:latest` → **rien à pousser sur Docker Hub**.
- Ajouter le même bloc de service dans `docker-compose-prod.yml` , mais **bind sur localhost** pour la sécurité : `ports: - "127.0.0.1:8082:80"` puis y accéder par tunnel SSH (`ssh -L 8082:localhost:8082 user@IP`).
- **Ne jamais exposer phpMyAdmin publiquement** sur le domaine.

Résumé express (à coller au camarade)

1. Installer Docker Desktop + le lancer (icône verte).
2. Récupérer le dossier du projet (git clone ou copie).
3. Terminal dans le dossier : `WEB_PORT=8088 docker compose up -d --build`
4. Attendre ~3 min, puis ouvrir `http://localhost:8088`
5. Se connecter avec un compte de test.

✅ Pas besoin d'installer Go, MySQL, ni WAMP : **tout est dans les conteneurs**. Le seul prérequis est **Docker Desktop**.